reg no: 241501223
**name: Sunil Varma V**

| **Exp No: 13** | **Implement your own ping program** |
|---|---|

## AIM:

To implement our own ping program using raw sockets in Python, which sends ICMP Echo Request messages to a host, receives the ICMP Echo Replies and displays the round trip time and the packet loss.

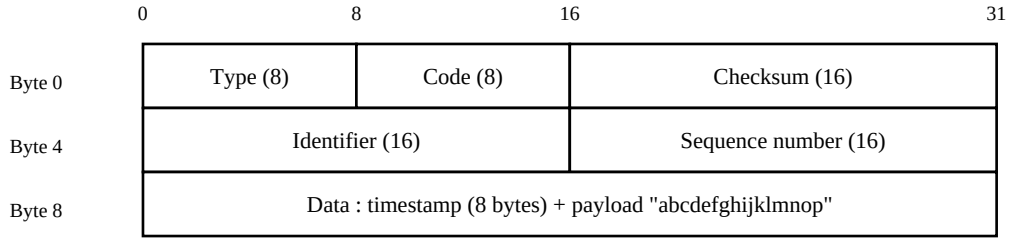
## ALGORITHM / PROCEDURE:

1. Read the host name given in the command line and convert it into an IP address using `socket.gethostbyname()`.
2. Create a **raw socket** using `socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)`. Raw sockets need administrator rights, so the program is run with `sudo`.
3. Build the ICMP Echo Request: **Type** = 8, **Code** = 0, **Checksum** = 0 (for now), **Identifier** = process id, **Sequence number** = 1, 2, 3 ... followed by the data (current time + 16 bytes of text).
4. Calculate the **Internet checksum** (16 bit one's complement sum of the whole ICMP message) and put it in the checksum field.
5. Send the packet with `sendto()` and note the time. Wait for the reply with `recvfrom()` (time out = 2 seconds).
6. The received data has a 20 byte IP header in front. Skip it, read the ICMP header and check that the **type** is 0 (Echo Reply) and the **identifier** is our id. Read the TTL from byte 8 of the IP header.
7. Round trip time = time of receiving - time of sending. Display the reply line. If no reply comes, display `Request timed out`.
8. Repeat steps 3 to 7 four times with a gap of one second.
9. Finally display the number of packets sent, received, % loss and the minimum / average / maximum round trip time.

### Header and functions used

| Item | Meaning |
|---|---|
| `struct.pack("!BBHHH", ...)` | Packs type, code, checksum, identifier and sequence into 8 bytes in network byte order |
| `socket.SOCK_RAW` | Gives direct access to the IP packets (we build the ICMP part ourselves) |
| `IPPROTO_ICMP` | Tells the kernel that this raw socket handles ICMP messages (IP protocol number 1) |
| `checksum()` | Adds all 16 bit words, adds the carry back and takes the one's complement |
| `data[8]` | TTL value inside the received IP header |

# DIAGRAM:




*Echo Request: Type = 8, Code = 0* *Echo Reply: Type = 0, Code = 0*

**Fig 1: ICMP Echo Request / Reply message format**

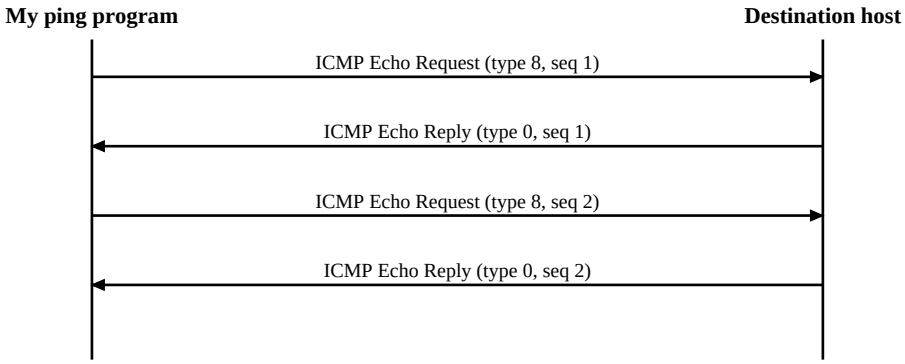



**Fig 2: Working of the ping program**

# PROGRAM:

**myping.py**

```
# Own ping program using RAW sockets (ICMP) - run as root
import socket, struct, time, sys, os

def checksum(data):
    if len(data) % 2:
        data += b'\x00'
    s = sum(struct.unpack("!%dH" % (len(data) // 2), data))
    s = (s >> 16) + (s & 0xffff)
    s += s >> 16
    return ~s & 0xffff

def make_packet(seq, pid):
    header = struct.pack("!BBHHH", 8, 0, 0, pid, seq)      # type 8 = echo request
    payload = struct.pack("!d", time.time()) + b"abcdefghijklmnop"
    chk = checksum(header + payload)
    header = struct.pack("!BBHHH", 8, 0, chk, pid, seq)
    return header + payload

def ping(host, count=4):
    try:
        ip = socket.gethostbyname(host)
    except socket.gaierror:
        print(f"ping: {host}: Name or service not known")
        return
    s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
    s.settimeout(2)
    pid = os.getpid() & 0xffff
    print(f"PING {host} ({ip}) 32 bytes of data.")
    times = []
    for seq in range(1, count + 1):
        s.sendto(make_packet(seq, pid), (ip, 0))
        t1 = time.time()
        try:
            while True:
                data, addr = s.recvfrom(1024)
                typ, code, _, rid, rseq = struct.unpack("!BBHHH", data[20:28])
                if typ == 0 and rid == pid:
                    break
            rtt = (time.time() - t1) * 1000
            ttl = data[8]
            times.append(rtt)
            print(f"{len(data)-20} bytes from {addr[0]}: icmp_seq={rseq} ttl={ttl} time={rtt:.3f} ms
              ")
        except socket.timeout:
            print(f"Request timed out for icmp_seq={seq}")
        time.sleep(1)
    s.close()
    lost = count - len(times)
    print(f"\n--- {host} ping statistics ---")
    print(f"{count} packets transmitted, {len(times)} received, {lost*100//count}% packet loss")
    if times:
        print(f"rtt min/avg/max = {min(times):.3f}/{sum(times)/len(times):.3f}/{max(times):.3f} ms")

ping(sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1")
```

## OUTPUT:

```
USER @USER -VirtualBox: ~
USER   @USER   -VirtualBox:~$ sudo python3 myping.py 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 32 bytes of data.
32 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms
32 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.017 ms
32 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.023 ms
32 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.036 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max = 0.015/0.023/0.036 ms
USER   @USER   -VirtualBox:~$ sudo python3 myping.py abc.invalid
ping: abc.invalid: Name or service not known
```

| Quantity | Value |
|---|---|
| Packets transmitted | 4 |
| Packets received | 4 |
| Packet loss | 0% |
| Minimum / average / maximum RTT | 0.015 / 0.023 / 0.036 ms |
| TTL of reply | 64 (loopback address) |

## RESULT:

Thus our own ping program was implemented using raw sockets. ICMP Echo Request packets were sent, the Echo Replies were received and the round trip time and packet loss statistics were displayed successfully.